Combat unit inheritance console

Chapter 3 · Practical Lab 1 · Inheritance, access and conversion

In this practical, you will build two simple game-unit classes and discover what inheritance actually gives you. You will construct an Archer from a CombatUnit, compare a copied base object with a base view, and recover a derived pointer only when the object is known.

You will write the member-function definitions yourself. The lab gives you the interfaces, small code fragments and exact console results, but it deliberately leaves the class logic and most printing statements for you to complete.

Duration: 120 minutes · C++17 · Visual Studio 2026 · Start from an Empty Project

> Working approach. Read one task, write the requested code, build, compare with the expected console output, and only then continue. Work from the most recent working version; do not rewrite the whole program at every step.

Before you begin

You need a Windows PC with Visual Studio and the Desktop development with C++ workload. Use Platform Toolset v145, ISO C++17, warning level /W4, and Debug / x64. You should already know basic C++ classes, private members, constructors, const member functions, references and pointers from Chapters 1–2.

What you will practise

- recognising a useful “is-a” relationship and writing public inheritance;
- initialising the base class before the derived object’s members;
- using a base class’s public functions without accessing its private data;
- reading constructor and destructor order from a console trace;
- distinguishing a copied base object from a pointer or reference view;
- explaining why a non-virtual call through a base view selects the base function; and
- using a downcast only when the complete object is already known.

Not required in this practical

No virtual functions, override, dynamic allocation, smart pointers, inheritance-based deletion, dynamic_cast, complex validation, new game systems or student-created test framework. These belong to later work. The focus is the class relationship itself.

The problem

A small game uses ordinary CombatUnit objects and specialised Archer objects. Every CombatUnit has health. An Archer is also a CombatUnit, but adds an arrow-attack value. The goal is to model that relationship clearly, rather than copy the health field into a second unrelated class.

For this exercise, the base attackPower() returns 0 and the Archer's same-named function returns its arrow-attack value. Both functions are deliberately non-virtual, just like the introductory lecture. This lets you observe an important difference between the type of an expression and the complete object it refers to.

Files you will create

File	Purpose
Main.cpp	Create objects, call public functions and print labelled results.
CombatUnit.h	Declare the base class, its constructor, queries and private health.
CombatUnit.cpp	Define the base operations and its constructor/destructor messages.
Archer.h	Declare a class that publicly derives from CombatUnit.
Archer.cpp	Define the derived constructor, queries and destructor message.

Your two-hour route

Stage	What you will do	Time
01	Create and configure the Visual Studio project	12 min
02	Define the CombatUnit base and implement its methods	20 min
03	Derive Archer and implement its behaviour	25 min
04	Compare direct calls, slicing and base views	22 min
05	Use a safe known-object upcast and downcast	15 min
06	Investigate public and protected access	10 min
07	Complete the output, review and save evidence	16 min

01 · Create the project from scratch

In Visual Studio, choose Create a new project, then Empty Project for C++. Name the project CombatUnitInheritanceLab. Use the same name for the solution if Visual Studio asks. The project should start with no C++ source files.

Choose Debug and x64 in the toolbar. Open Project → Properties and, for All Configurations, select Platform Toolset v145, C++ Language Standard ISO C++17, and Warning Level Level 4 (/W4). Keep the project as a Console application and leave precompiled headers disabled.

Under Source Files, add Main.cpp. Start with this small complete program:

```
#include <iostream>

int main()
{
    std::cout << "Combat unit inheritance lab\n";
    return 0;
}
```

Build using Ctrl+Shift+B, then run using Ctrl+F5. The only expected line is:

```
Combat unit inheritance lab
```

Now add two header files (CombatUnit.h and Archer.h) and two source files (CombatUnit.cpp and Archer.cpp). They will initially be empty. Keep one main() function in the entire project.

> Checkpoint 01. The untouched starter builds and prints its heading. If you cannot build, resolve the project configuration before attempting inheritance.

02 · Build the CombatUnit base class

Open CombatUnit.h. This class stores one health value and offers three operations: read health, report a basic attack value, and construct a unit with a starting health. Type the interface below. This is the declaration; you will create the function bodies yourself in CombatUnit.cpp.

```
#ifndef COMBAT_UNIT_H
#define COMBAT_UNIT_H

class CombatUnit
{
public:
    explicit CombatUnit(int t_health);
    ~CombatUnit();
    int health() const;
    int attackPower() const;

private:
    int m_health;
};

#endif
```

Write the definitions in CombatUnit.cpp

Start by including your own header and <iostream>. Write one definition for each function. Use the scope-resolution operator to show that each definition belongs to CombatUnit.

Definition	What it must do
CombatUnit(int t_health)	Use a member initialiser list to give m_health the supplied value; print + CombatUnit.
~CombatUnit()	Print - CombatUnit when this object is destroyed.
health() const	Return the object's current m_health.
attackPower() const	Return 0; the base class is a generic unit.

> Hint. The constructor initialiser list comes after the function header and before the body: : m_health{t_health}. The destructor has a ~ before the class name and no return type. Every definition must match its header declaration, including const.

Back in Main.cpp, include CombatUnit.h. After the heading, create a small inner block containing CombatUnit guard{60};. Print its health and attack power using its public functions. Put the label Base unit before the block.

Expected console section

```
Base unit
+ CombatUnit
Health: 60
Attack: 0
- CombatUnit
```

The minus line appears when the closing brace ends the local object's lifetime. You do not call the destructor manually.

> Checkpoint 02. The base class builds and reports 60 and 0. Its constructor and destructor messages surround the two output lines.

03 · Make Archer a specialised CombatUnit

An Archer is a CombatUnit with one extra piece of information: arrow-attack power. You do not need a second health field. The inherited CombatUnit already stores health and provides health().

Write Archer.h

Include CombatUnit.h because a derived class needs the complete definition of its base. Then add the following declaration:

```
#ifndef ARCHER_H
#define ARCHER_H

#include "CombatUnit.h"

class Archer : public CombatUnit
{
public:
    Archer(int t_health, int t_arrowPower);
    ~Archer();
    int arrowPower() const;
    int attackPower() const;

private:
    int m_arrowPower;
};

#endif
```

Implement Archer.cpp yourself

Include Archer.h and <iostream>. Write the definitions in this order: constructor, destructor, arrowPower(), attackPower(). You already completed the corresponding pattern for CombatUnit, so use that work as a guide rather than copying a whole solution.

Operation	Your implementation task
Archer constructor	Initialise the direct CombatUnit base with t_health. Initialise m_arrowPower with t_arrowPower. Then print + Archer.
Archer destructor	Print - Archer. Do not manually call the CombatUnit destructor.
arrowPower() const	Return the Archer's own arrow-attack member.
attackPower() const	Return the same arrow-attack value. This deliberately hides a non-virtual base function with the same name.

> Hints. Start the constructor initialiser list with CombatUnit{t_health}. Then initialise m_arrowPower. The base is constructed before the derived members and constructor body. You may call the inherited health() function; you may not directly access CombatUnit's private m_health.

In Main.cpp, add another labelled inner block with Archer archer{80, 12};. Print health and attack power from the Archer. Predict the order of the plus and minus messages before running.

Expected console section

```
Archer unit
+ CombatUnit
+ Archer
Health: 80
Attack: 12
- Archer
- CombatUnit
```

04 · Compare a copy with two views

This is the most important reasoning stage. You will construct one complete Archer, copy only its CombatUnit part into a separate object, and then create two ways to view the original without copying it.

Add a new section named Copy and views. Put these declarations inside a fresh inner block in Main.cpp:

```
Archer archer{80, 12};
CombatUnit baseCopy{archer};
CombatUnit const& baseView{archer};
CombatUnit const* pBase{&archer};
```

Now write the cout statements yourself. In this order, print the attack power reported by the original Archer, baseCopy, baseView and pBase. Finally, print the original Archer's attack power once more. Use the labels Direct attack, Copied attack, Reference attack, Pointer attack and Original attack.

Expression	What it reads	Expected
archer.attackPower()	Direct Archer interface	12
baseCopy.attackPower()	Separate CombatUnit object	0
baseView.attackPower()	CombatUnit reference to Archer	0
pBase->attackPower()	CombatUnit pointer to Archer	0
archer.attackPower() again	Original Archer, unchanged	12

The copy is an independent CombatUnit: its Archer-only state is not part of the destination. The reference and pointer still refer to the complete Archer, but the call uses the non-virtual CombatUnit version because their declared access type is CombatUnit.

Expected console section

```
Copy and views
+ CombatUnit
+ Archer
Direct attack: 12
Copied attack: 0
Reference attack: 0
Pointer attack: 0
Original attack: 12
- CombatUnit
- Archer
- CombatUnit
```

Notice the three minus messages. baseCopy is a real CombatUnit and is destroyed first. Its copy construction used an implicitly generated copy constructor, so it did not print an extra + CombatUnit message. Then the complete Archer destroys, followed by its CombatUnit base part.

Checkpoint 04. Explain in your own words why a reference can avoid slicing and still print the base attack value. Do not add virtual to "fix" the output; that is part of the next lecture.

05 · Use a safe known-object conversion

Public inheritance lets a CombatUnit pointer refer to the base part of an existing Archer. The pointer does not own the Archer, and creating it does not allocate anything.

Add an inner block headed Known-object cast. Start with these declarations:

```
Archer archer{75, 9};
CombatUnit* pUnit{&archer};
Archer* pRecovered{static_cast<Archer*>(pUnit)};
```

Write two labelled cout statements: read health through pUnit, then read attack power through pRecovered. Here the downcast is valid because you just created an Archer and assigned its address to pUnit. The cast does not check that type at runtime; in this particular block you already know it.

Expected console section

```
Known-object cast
+ CombatUnit
+ Archer
Base health: 75
Recovered attack: 9
- Archer
- CombatUnit
```

Safety rule. Do not use static_cast<Archer*> on an arbitrary CombatUnit pointer. An ordinary CombatUnit is not secretly an Archer, and a non-null result would not prove that it is. Never delete either pointer in this lab: both only borrow access to the local archer.

Checkpoint 05. You can explain both conversions: Archer* to CombatUnit* is a public upcast, while the reverse static_cast is justified only by the known object in this block.

06 · Check what an access change means

You have been using public inheritance. That means other code can use the Archer through the public CombatUnit interface. Now examine what would happen if the relationship were declared protected instead.

First predict the result; you do not need to change the program to answer it. If you have time, make this one controlled build-only change in Archer.h:

```
class Archer : protected CombatUnit
```

Build once and look at the first useful access error. In Main.cpp, an unrelated caller can no longer use inherited public health() directly or implicitly form a CombatUnit* from an Archer*. The Archer class can still use suitable inherited public and protected operations inside its own functions.

Restore class Archer : public CombatUnit and rebuild before continuing. Remember that changing public to protected did not remove the CombatUnit base part, and it did not make the base's private m_health directly accessible.

Checkpoint 06. Your final project must use public inheritance and compile successfully. Only the access rule changed during this investigation; the object still contains its CombatUnit base part.

07 · Put Main.cpp into its final form

Keep the Main.cpp that you have been developing. Do not replace it with a long completed driver. The outline below contains the object declarations and important relationships; add the section headings and the required cout statements yourself using the expected outputs on the following page.

```
// Keep #include <iostream> and your class headers.
// Inside main(), after the lab heading:

// Base unit: a local CombatUnit.
{
    CombatUnit guard{60};
    // Print health and base attack power.
}

// Archer unit: base construction plus derived state.
{
    Archer archer{80, 12};
    // Print inherited health and Archer attack power.
}

// Copy and views: preserve the original Archer.
{
    Archer archer{80, 12};
    CombatUnit baseCopy{archer};
    CombatUnit const& baseView{archer};
    CombatUnit const* pBase{&archer};
    // Print five attack results, in the stated order.
}

// Known-object cast: no allocation and no delete.
{
    Archer archer{75, 9};
    CombatUnit* pUnit{&archer};
    Archer* pRecovered{static_cast<Archer*>(pUnit)};
    // Print base health and recovered Archer attack.
}
```

Keep every set of braces: each block gives its local objects a clear end of lifetime. Build and run the program again after restoring any temporary access changes.

Complete expected console output

```
Combat unit inheritance lab

Base unit
+ CombatUnit
Health: 60
Attack: 0
- CombatUnit

Archer unit
+ CombatUnit
+ Archer
Health: 80
Attack: 12
- Archer
- CombatUnit

Copy and views
+ CombatUnit
+ Archer
Direct attack: 12
Copied attack: 0
Reference attack: 0
Pointer attack: 0
Original attack: 12
- CombatUnit
- Archer
- CombatUnit

Known-object cast
+ CombatUnit
+ Archer
Base health: 75
Recovered attack: 9
- Archer
- CombatUnit
```

If something does not work

What you see	What to check first
A constructor needs arguments	Did Archer directly initialise CombatUnit in its constructor initialiser list?
Unresolved external symbol	Does every declared function have a matching .cpp definition and const suffix?
Cannot access m_health	It is private in CombatUnit. Use inherited health() instead.
Copy/reference attacks unexpectedly show 12	Check the actual type of the copied object and make sure attackPower remains non-virtual.
Wrong destructor order or extra traces	Check inner braces and whether you made additional objects/copies.
Archer* cannot convert to CombatUnit*	Restore public inheritance after the access investigation.

Finish and record your evidence

Complete the short Lab_01_Evidence_Template_Combat_Unit.md alongside this practical. It asks for the outputs you already produced and a few brief explanations—not another test suite. Submit or save the complete Visual Studio solution/project, all source files, and the evidence in the manner specified by your lecturer.

> Final check. Your project builds in Debug / x64 with /W4; all temporary build-only changes are restored; every output line matches the published transcript; the base members remain private; and no base pointer is deleted.

Optional stretch · Add a third inheritance level

Only attempt this after the core lab works. Add VeteranArcher : public Archer. Give it one private integer m_bonus, a constructor that initialises Archer first and then m_bonus, and a non-virtual attackPower() const that adds the bonus to the Archer attack.

For an Archer attack of 12 and bonus 3, the VeteranArcher should report attack power 15. Use Archer::attackPower() to request the base calculation deliberately, rather than calling attackPower() unqualified inside itself. This is a separate experiment; do not change the core final output.